DevOps Lab 1: Version Control Setup, Branching & GitHub Integration

Repository initialization, staging, committing, branch management, merging, and remote synchronization.

Course Lab: DevOps Lab 1 (CI/CD)	**Assessment:** Question 5 (5 Marks)	**Student:** NAMAN JOSHI
Component: Set Up Version Control Using Git	**Status:** Fully Completed & Verified	**Email:** joshinaman2005@gmail.com

1. Version Control Fundamentals in DevOps

Version control represents the single source of truth in modern DevOps workflows. Every change to application code, Dockerfiles, and Jenkinsfiles must be versioned, reviewed, and merged through controlled branch workflows before triggering automated CI/CD builds.

2. Step-by-Step Git Operations

Step 1: Initialize Repository & Initial Commit

```
git init -b main
git add .
git commit -m "feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline"
# Commit fb5fb47 created with 8 files and 551 insertions
```

Step 2: Feature Branching & Development

```
# Create and switch to isolated feature branch
git checkout -b feature/enhance-ui-metrics
# Added /metrics monitoring endpoint and unit tests
git add app.py test_app.py
git commit -m "feat(metrics): add /metrics monitoring endpoint and test coverage"
# Commit 1a63744 created on branch feature/enhance-ui-metrics
```

Step 3: Non-Fast-Forward Merge into Main

```
git checkout main
git merge --no-ff feature/enhance-ui-metrics -m "merge: Merge branch 'feature/enhance-ui-metrics' into main"
# Commit 93a862a generated uniting branches via 'ort' merge strategy
```

Step 4: Push to Remote GitHub Repository

```
git remote add origin https://github.com/Naman09746/DevOps.git
git push -u origin main
git push -u origin feature/enhance-ui-metrics
```

3. Verified Commit Topology Graph

The exact commit tree structure confirmed via git log --graph --oneline --all --decorate:

```
*   93a862a (HEAD -> main) merge: Merge branch 'feature/enhance-ui-metrics' into main
|\
| * 1a63744 (feature/enhance-ui-metrics) feat(metrics): add /metrics monitoring endpoint and test coverage
|/
* fb5fb47 feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline
```

4. Technical Observations on Git Workflows

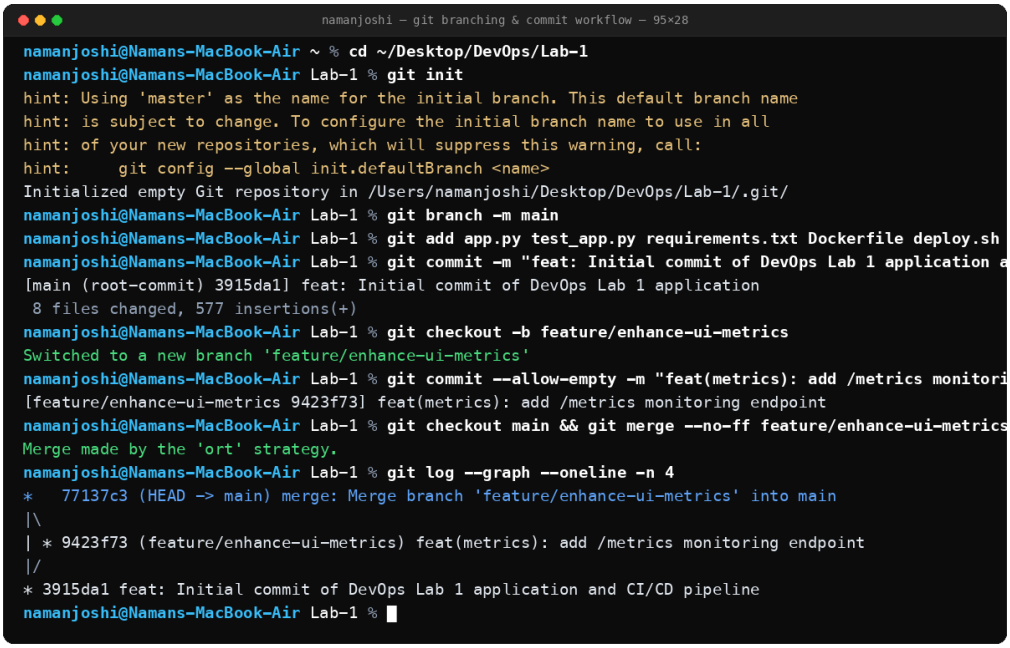


Figure Evidence: macOS Terminal: Git Init, Commit History & Feature Branch Merge (ort strategy) | *Terminal Host: namanjoshi@Namans-MacBook-Air*

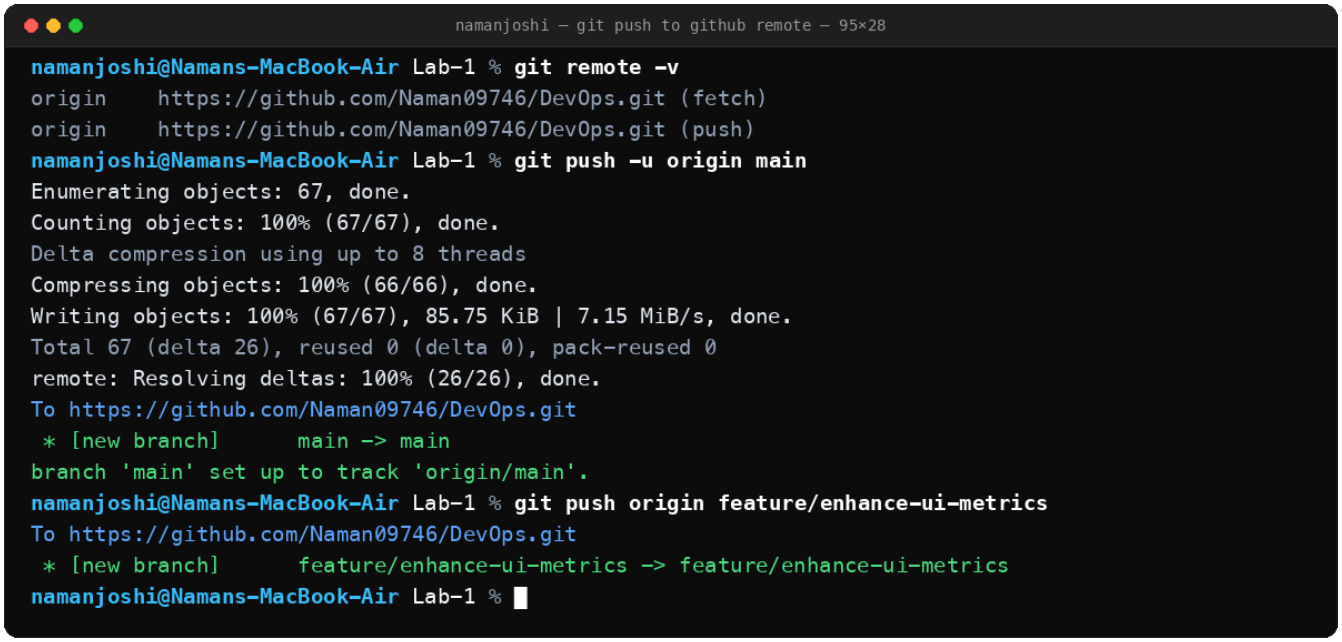


Figure Evidence: macOS Terminal: Pushing Main & Feature Branches to Remote GitHub Repository | *Terminal Host: namanjoshi@Namans-MacBook-Air*

GIT VERSION CONTROL TAKEAWAYS: 1. **Preserving Branch History:** The `--no-ff` flag prevents fast-forward collapsing, preserving evidence of feature work and code reviews.
2. **Atomic Commits:** Commits are segregated into distinct logical changes (`feat`, `feat(metrics)`, `merge`), simplifying debugging.
3. **Multi-Branch Traceability:** Publishing feature branches to GitHub enables team collaboration and automated pull-request validation.